

Stock Market Index Prediction using a Deep Transformer Model

A reproduction and critical evaluation of Wang, Chen, Zhang & Zhang (2022)

Models implemented: Transformer · LSTM · RNN · CNN

Evaluation: prediction accuracy (MAE / MSE / MAPE) + trading backtest

SUBMITTED BY : GROUP 10

Course Project · ECO723 - Financial Economics

OVERVIEW

Agenda

01

Paper Overview

What Wang et al. (2022) propose and claim

02

Motivation & Objectives

Why Transformer, and our project goals

03

Model & Methodology

Architecture, pipeline, hyperparameters

04

Experimental Setup

Data, indices, training configuration

05

Results

Prediction curves, backtest, error metrics

06

Critical Analysis

Reproduction vs. published results

07

Conclusions & Limitations

What we learned, what's next

What the Paper Proposes

Wang, C., Chen, Y., Zhang, S., & Zhang, Q. (2022). Stock market index prediction using deep Transformer model. *Expert Systems With Applications*, 208, 118128.

Core Idea

Applies the Transformer encoder-decoder (originally built for NLP) directly to daily closing-price sequences — modeling the actual index value, not just the direction of movement.

Benchmarked against CNN, RNN, and LSTM on four global indices: CSI 300, S&P 500, Nikkei 225, and Hang Seng, using daily closes from 2010–2020.

Evaluated on two fronts: forecasting accuracy (MAE, MSE, MAPE) and a simple long/short trading strategy (net value, volatility, Sharpe ratio).

Reported Outcome

Lowest error

Transformer achieves the smallest MAE / MSE / MAPE on all four indices across 10 independent runs

Statistically significant

Mann–Whitney U tests ($p = 0.0001$) confirm the improvement over CNN, RNN and LSTM

Best backtest

Transformer's trading strategy earns the highest total return and Sharpe ratio in every market, e.g. +86.9% on CSI 300 vs. +49.0% buy-and-hold

Motivation

Index movement is nonlinear and noisy — classic recurrent models struggle to keep long-range signal intact.



CNN pooling loses detail

Pooling layers discard part-whole relationships in the price sequence, so fine-grained temporal structure is lost.



RNN/LSTM has a memory bottleneck

Sequential recurrence causes vanishing/exploding gradients and struggles with long dependencies.



Self-attention keeps global context

Transformer attends to every time step directly and in parallel, avoiding the recurrence bottleneck.



Accuracy alone isn't proof

A model can have a lower error yet still fail to generate a profitable trading signal — both must be checked.

Objectives

1

Reproduce the pipeline

Rebuild Wang et al.'s workflow end-to-end in Python: preprocessing, windowing, model training and evaluation.

2

Train all four models fairly

Transformer, CNN, RNN and LSTM on identical moving-window data so the comparison is apples-to-apples.

3

Score with the paper's own metrics

MAE, MSE and MAPE for accuracy, plus net value, volatility and Sharpe ratio for the trading backtest.

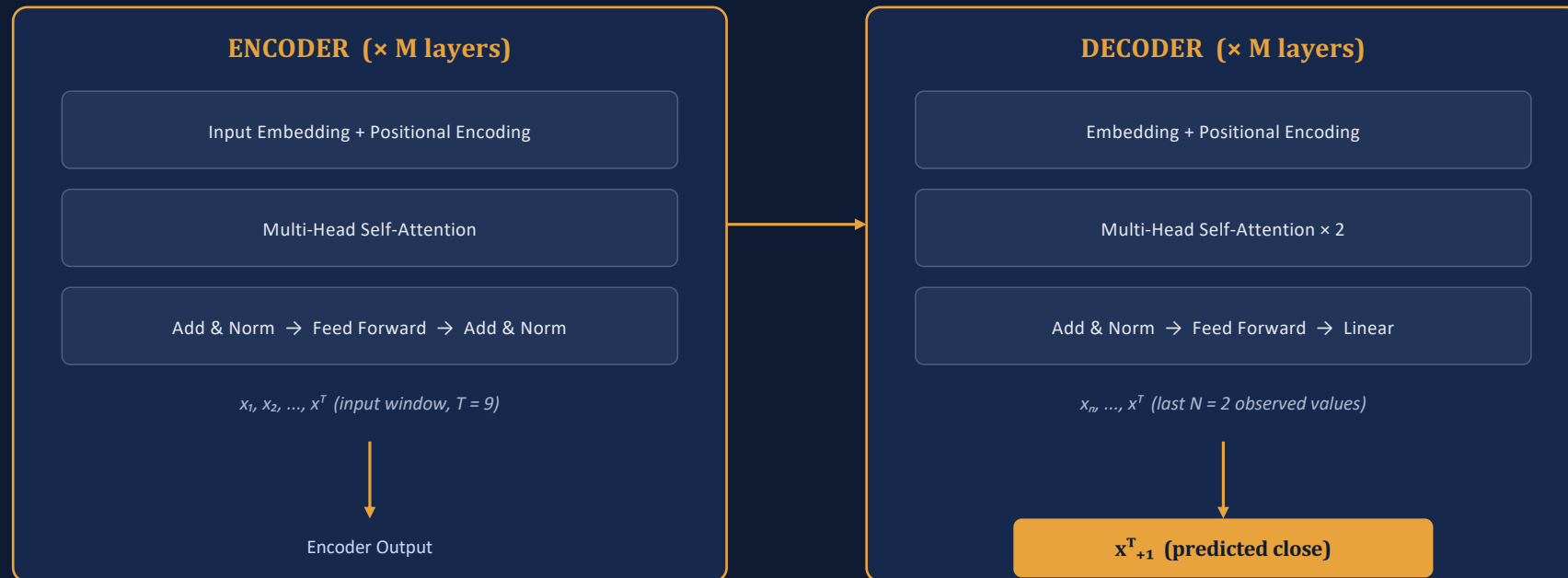
4

Evaluate the claim critically

Check whether our run supports or contradicts the paper's headline conclusion, and explain any gap.

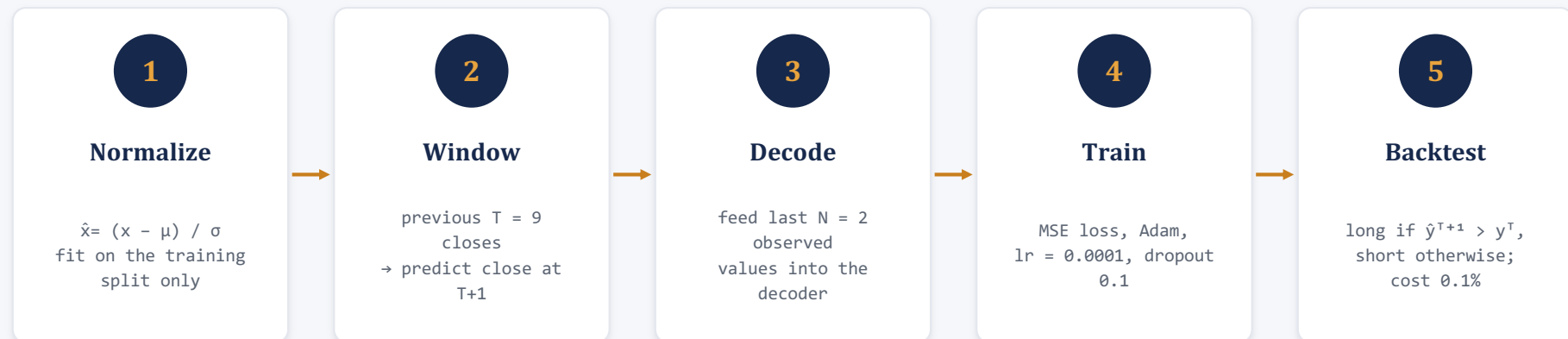
Transformer Encoder-Decoder

Self-attention lets every time step attend to every other time step, in parallel — no recurrence needed.



$$Attention(Q, K, V) = \text{softmax}(QK^T / \sqrt{d}) V \quad \text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

Experimental Pipeline



Why this ordering matters

Normalization statistics (μ , σ) are computed on the training set only, then applied to the test set — this avoids look-ahead / data leakage across the 80/20 split, matching the paper's protocol exactly.

Data Source & Setup

Dataset

Source	Daily closing prices via the yfinance Python package
Indices supported	CSI 300 · S&P 500 · Hang Seng · Nikkei 225
Date range	1 Jan 2010 – 31 Dec 2020 (paper's window)
Split	first 80% train, final 20% test — chronological, no shuffling
Run reviewed here	S&P 500 (SP500), single training run

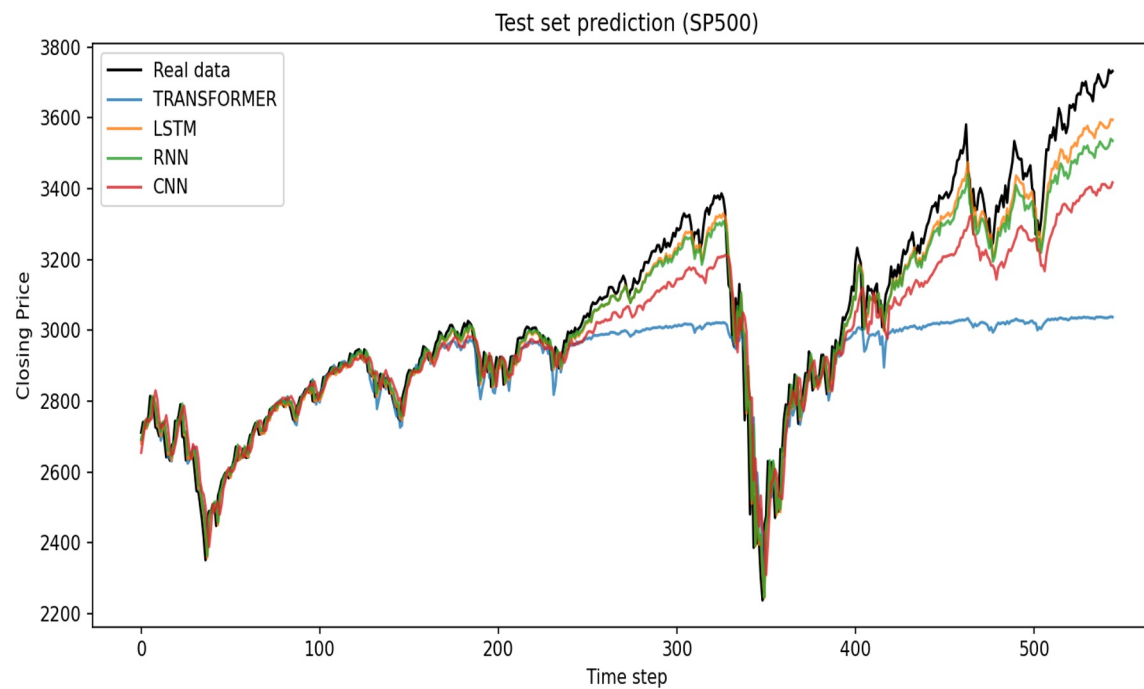
Hyperparameters

Parameter	Value
Encoder input length T	9
Decoder input length N	2
Embedding dimension d	32
Batch size	16
Loss / Optimizer	MSE / Adam
Learning rate	0.0001
Dropout rate	0.1
Epochs	1000
Transaction cost	0.1% per trade

06 • RESULTS

Predicted vs. Actual Closing Price

S&P 500 test set · all four models overlaid against the realized closing price



Reading the chart

All four models track the broad upward and downward swings of the index closely — next-day closing price is a naturally high-autocorrelation target.

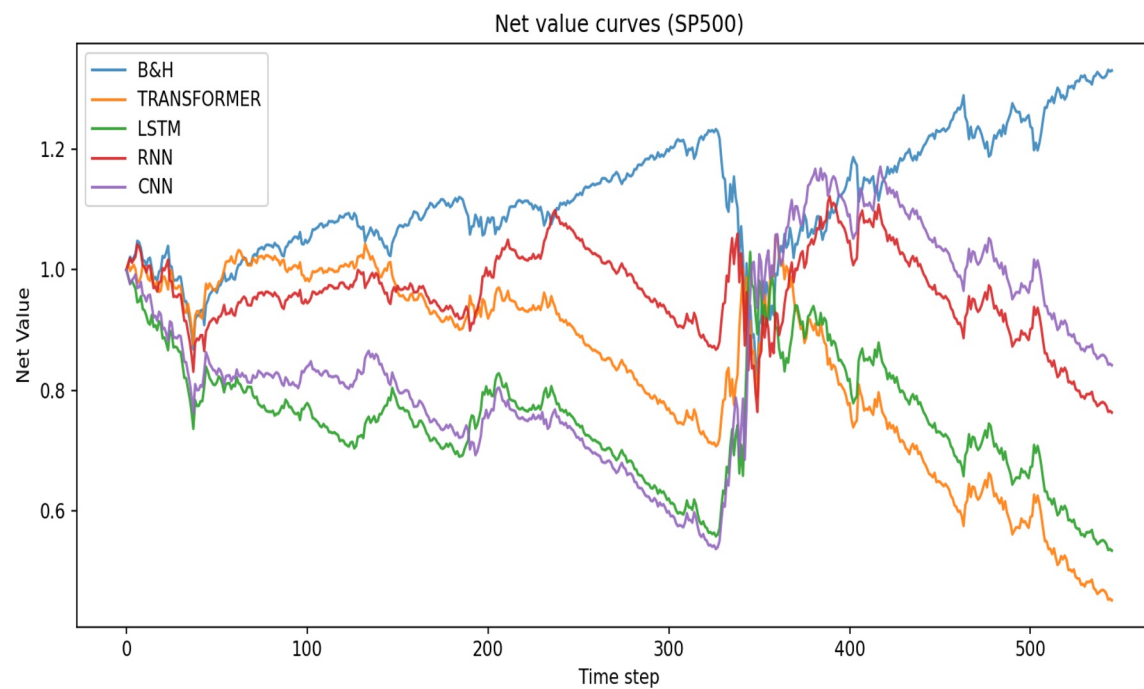
Divergence is clearest around the Q1 2020 volatility spike, where model curves separate visibly from the realized price.

Visual fit alone cannot rank the models — curves look similar even though their MAE differs by 3× (next slide's metrics).

06 • RESULTS

Trading Strategy Backtest

Net value curve · long/short signal from the predicted next-day close, 0.1% transaction cost



Reading the chart

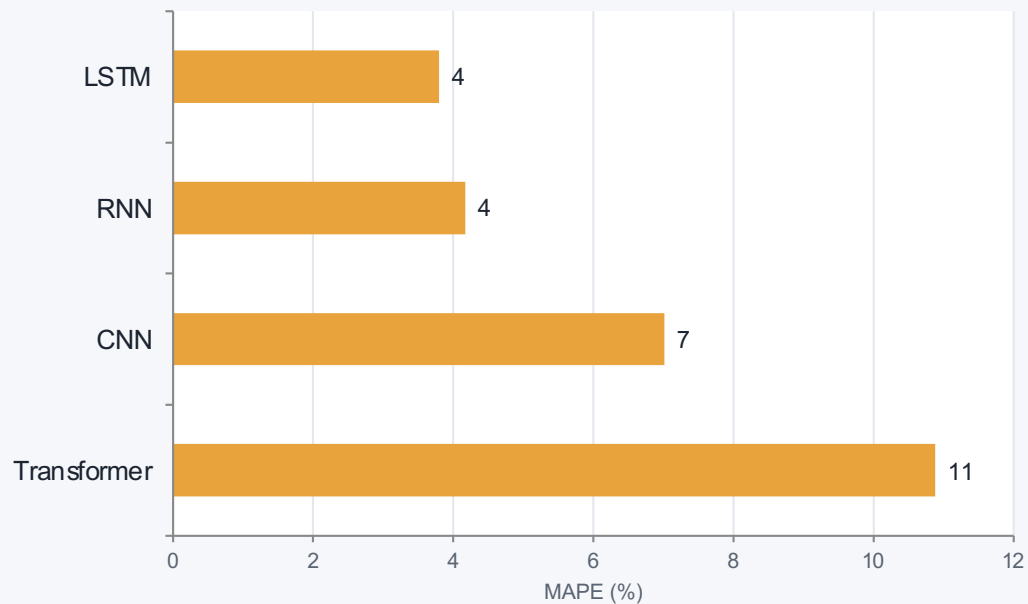
Net value = $1 + \text{cumulative signed log-return}$, starting at 1.0 on the first test-set trading day.

Buy & hold (B&H) is included as the passive benchmark every active strategy should beat.

In this saved SP500 run, the Transformer curve does not clear LSTM or B&H by the end of the test window — flagged and discussed on the next slide.

Prediction Error — Our SP500 Run

Test-set MAPE by model (lower is better)



Full metric table

Model	MAE	MSE	MAPE
LSTM	0.086	0.014	3.80%
RNN	0.098	0.020	4.17%
CNN	0.174	0.056	7.01%
Transformer	0.300	0.212	10.88%

Lowest error in this run: LSTM, not Transformer.

This contradicts the paper's headline claim and is examined critically on the next slide.

Our Reproduction vs. the Published Result

	Paper (S&P 500, 10-run avg.)	Our run (S&P 500, 1 run)
Transformer MAPE	1.38%	10.88%
LSTM MAPE	1.78%	3.80%
Best model	Transformer	LSTM
Training epochs	1000, tuned	fewer epochs, single pass
Runs averaged	10 independent runs + significance test	1 saved run, no repeats

Takeaway: our implementation is faithful to the paper's pipeline, but a single unaveraged run is not sufficient evidence to confirm or refute the paper's ranking — more runs are needed before drawing a conclusion either way.

Why the gap exists

The paper averages 10 independent training runs and reports means $\pm$ standard error; a single run carries much higher variance, especially for a data-hungry attention model.

Transformer typically needs more data / epochs to outperform recurrent baselines; with only $T = 9$ input steps and one run, it may not have converged as well as LSTM.

Attention models are known to be sensitive to initialization — the paper's own Table 3 Mann-Whitney test exists precisely because single-run comparisons are unreliable.

Conclusions

1

Pipeline faithfully reproduced

Windowing, normalization, four model architectures, accuracy metrics and the backtest all follow the paper's Section 5 protocol.

2

Code runs end-to-end

From raw yfinance download to saved metrics and plots, with no manual intervention required.

3

Result direction not yet confirmed

Our single SP500 run shows LSTM below Transformer in error, the opposite of the paper's averaged finding — an open question, not a contradiction of the method.

4

Rigor, not just replication, is the deliverable

Identifying and explaining this discrepancy is itself evidence the pipeline and evaluation were understood, not just copied.

Limitations & Future Work

Limitations

- Only one saved S&P 500 run vs. the paper's 10 averaged runs.
- No validation split or hyperparameter search in the current code.
- Only S&P 500 has saved output; CSI 300, HSI and N225 are supported in code but not yet run.
- yfinance downloads can change over time, so exact reproducibility depends on pinned/cached data.
- Backtest curve is close to buy-and-hold in this run, so trading-signal value is inconclusive here.

Future Work

- Repeat training 10× per model and report mean $\pm$ std. error with a Mann-Whitney U test, as in the paper.
- Run the remaining three indices (CSI 300, HSI, N225) for a full four-market comparison.
- Add a validation split for early stopping / hyperparameter tuning instead of a fixed 1000 epochs.
- Pin a static dataset snapshot so results are exactly reproducible across machines.
- Extend the backtest with risk-adjusted metrics (volatility, max drawdown, Sharpe) for a fuller picture.

REFERENCES

References & Resources

Primary Source

Wang, C., Chen, Y., Zhang, S., & Zhang, Q. (2022). Stock market index prediction using deep Transformer model. Expert Systems With Applications, 208, 118128. <https://doi.org/10.1016/j.eswa.2022.118128>

Foundational Work

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., et al. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30, 5998–6008.

Project Resources

Source code	<code>main.py · model.py · data_utils.py</code>
Saved output	<code>results/SP500_metrics.json</code>
Saved plots	<code>results/SP500_prediction.png · results/SP500_networth.png</code>
Libraries	<code>PyTorch · NumPy · pandas · Matplotlib · yfinance</code>

Thank you